A COMPACT FIELD GUIDE FOR BUILDERS

AI-Native System Design

Die wenigen Ideen hinter modernen Cloud-, Daten-, Model- und Agent-Systemen. Von null bis zu globalen Cells - ohne sechshundert Seiten Umweg.

Grundidee	Systemdesign ist die Kunst, Grenzen und Failure Modes bewusst zu waehlen.
Lernmethode	Jedes Kapitel beginnt mit einer These und endet mit einem Entscheidungsmodell.
AI-native	Model Serving, GPU-Oekonomie, Retrieval, Evals und Agents sind First-Class-Systemteile.
Ziel	Nach dem Buch sollst du Systeme erklaeren, dimensionieren und kritisch vereinfachen koennen.

Datei: ai-native-system-design-field-guide.pdf

Erstellt am 2026-08-12 / Originale Urban Learning Edition.

TEIL 1 / FIRST PRINCIPLES

Zuerst das Problem, dann die Architektur

Die beste Architektur ist keine Sammlung moderner Produkte. Sie ist eine kleine Zahl begruendeter Entscheidungen unter realen Grenzen.

1. Was Systemdesign wirklich ist

Ein System ist eine Maschine fuer Versprechen.

Nutzer sehen keine Datenbank, Queue oder Kubernetes-Deployment. Sie sehen ein Versprechen: Die Nachricht kommt an, das Dokument bleibt privat, die Antwort erscheint schnell und ein bezahlter Auftrag geht nicht verloren. Systemdesign uebersetzt solche Produktversprechen in technische Invarianten.

Die wichtigste Arbeit passiert vor dem Diagramm. Welche Aussage muss immer wahr bleiben? Welche darf voruebergehend veraltet sein? Welcher Verlust ist tolerierbar? Was kostet eine falsche Antwort? Erst danach entscheidet man ueber Datenmodelle, Replikation, Caches und Services.

Ein Design ist gut, wenn seine Failure Modes verstaendlich sind. Eine Komponente ist nicht modern, weil sie verteilt ist. Sie ist nuetzlich, wenn sie eine konkrete Grenze verbessert und ihre neue Komplexitaet kleiner ist als das geloeste Problem.

Invariant	Eine Regel, die auch bei Fehlern nicht verletzt werden darf.
SLO	Ein messbares Versprechen ueber Zuverlaessigkeit oder Leistung.
Trade-off	Eine Verbesserung wird mit einer anderen Eigenschaft bezahlt.
Blast Radius	Der Teil des Systems oder der Kunden, den ein Fehler treffen kann.

2. Requirements vor Komponenten

> Functional Requirements sagen was; Quality Attributes sagen wie gut.

Functional Requirements beschreiben Faehigkeiten: Nutzer laden Dateien hoch, Agents rufen Tools auf oder Teams teilen Memory. Non-functional Requirements beschreiben die Qualitaet: Latenz, Verfuegbarkeit, Datenschutz, Konsistenz, Kosten und Betriebsaufwand.

Schreibe zuerst die drei kritischsten User Journeys. Fuer jede Journey definierst du Datenvolumen, Peak Rate, Antwortbudget, erlaubten Datenverlust und Auswirkungen eines Fehlers. Diese Tabelle ist wertvoller als ein fruehes Architekturdiagramm.

Die Anforderungen muessen priorisiert sein. Gleichzeitig globale starke Konsistenz, null Latenz, unbegrenzte Verfuegbarkeit und minimale Kosten sind keine Anforderungen, sondern ein Widerspruch. Senior Design macht die Reihenfolge explizit.

Correctness	Welche falsche Wirkung darf niemals entstehen?
Latency	Welches Ende-zu-Ende-Budget erleben Nutzer?
Availability	Wie viel Ausfall pro Monat ist akzeptabel?
Durability	Welche bestaetigten Daten duerfen verloren gehen?
Cost	Welche Einheit treibt Kosten: Request, Token, GPU-Sekunde oder Byte?

3. Last mit einer Serviette rechnen

> Architecture follows arithmetic.

Beginne mit aktiven Nutzern, nicht registrierten Accounts. Multipliziere aktive Nutzer mit Operationen pro Zeitraum, teile durch Sekunden und wende einen Peak-Faktor an. Trenne Reads, Writes, grosse Payloads und Hintergrundarbeit.

Speicherwachstum ist Rate mal Retention. Netzwerk ist Requests mal Payload. Bei AI kommen Input Tokens, Output Tokens, Context-Laenge, Embedding-Dimensionen und GPU-Zeit hinzu. Eine plausible Groessenordnung reicht anfangs; falsche Praezision hilft nicht.

Little's Law verbindet Durchsatz, Dauer und Gleichzeitigkeit: Concurrency ist ungefaehr Arrival Rate mal durchschnittliche Zeit im System. 5.000 Requests pro Sekunde bei 400 Millisekunden bedeuten etwa 2.000 gleichzeitig offene Requests.

MENTAL MODEL

```
active users x actions / time x peak factor = peak RPS
RPS x average latency                = concurrency
writes / day x bytes x retention     = stored data
tokens / request x requests           = model workload
```

4. Latenz ist ein Budget

Die langsamste Abhaengigkeit ist selten allein schuld; Budgets addieren sich.

Eine Anfrage kann Gateway, Auth, Cache, Datenbank, Retrieval, Model Queue und Streaming durchlaufen. Weise jeder Stufe ein Budget zu. Ein aeusserer Timeout muss groesser als seine inneren Timeouts sein, aber kleiner als das Client-Budget.

P50 beschreibt die Mitte, P95 und P99 den langsamen Rand. Durchschnittswerte verstecken Warteschlangen, Cold Starts und Hot Shards. Nutzer erleben ausserdem nicht nur Time to Last Byte. Bei AI sind Time to First Token und Inter-Token-Latenz eigene Produktmetriken.

Tail Latency verstaerkt sich in Fan-out-Systemen. Wenn ein Request zwanzig parallele Backends braucht, reicht ein langsamer Teil, um den ganzen Request zu verzoegern. Begrenze Fan-out, verwende Deadlines und liefere degradierte, aber korrekte Ergebnisse.

TTFT	Zeit bis zum ersten sichtbaren Model-Token.
ITL	Zeit zwischen gestreamten Tokens.
P99	99 Prozent der Messungen sind schneller als dieser Wert.
Deadline	Absoluter Zeitpunkt, nach dem Arbeit keinen Wert mehr hat.

5. Failure ist ein Zustand, kein Sonderfall

> Netzwerke koennen gleichzeitig funktionieren und unklar sein.

Ein Timeout bedeutet nicht, dass die Gegenseite nichts getan hat. Vielleicht wurde der Write committed und nur die Antwort ging verloren. Ein Prozess kann zwischen zwei Zeilen abstuerzen. Eine Region kann erreichbar sein, aber falsche oder extrem langsame Antworten liefern.

Entwirf fuer Partial Failure: Welche Abhaengigkeit darf ausfallen? Welche Antwort bleibt korrekt? Welche Arbeit wird wiederholt? Wo entstehen Duplikate? Wie erkennt ein Operator den Zustand? Gute Systeme haben einen expliziten Degradation-Pfad.

Fehlerwahrscheinlichkeit ist nicht das einzige Risiko. Blast Radius entscheidet, ob ein Fehler einen Request, einen Tenant, eine Cell oder die ganze Plattform trifft. Isolation ist oft wertvoller als ein immer komplexerer globaler Failover.

MENTAL MODEL

```
normal path       -> full result
dependency slow   -> deadline + partial result
dependency down   -> cached or queued result
uncertain write   -> idempotent retry
regional failure  -> isolated failover decision
```

TEIL 2 / COMPUTE AND NETWORK

Was unter jedem Service passiert

Threads, Tasks, TCP, DNS und Routing sind keine Trivia. Sie erklaeren, warum Systeme unter Last warten, blockieren oder ihre eigene Kapazitaet verlieren.

6. Prozess, Thread und async Task

> Concurrency ist eine Scheduling-Frage; Parallelism eine Hardware-Frage.

Ein Prozess besitzt Speicher und Betriebssystemressourcen. Threads teilen den Prozessspeicher, haben aber eigene Stacks. Async Tasks sind kleinere Zustandsmaschinen, die auf Runtime-Threads geplant werden und beim Warten auf I/O pausieren koennen.

I/O-bound Arbeit profitiert von async, weil ein Thread viele offene Operationen verwalten kann. CPU-bound Arbeit braucht echte CPU-Zeit und muss durch Threads, Prozesse oder Accelerator-Worker parallelisiert werden. Blockierende Arbeit auf async Runtime-Threads erzeugt Head-of-Line Blocking.

Ein Service skaliert nicht nur durch mehr Threads. Locks, Datenbankverbindungen, Memory Bandwidth und gemeinsame Queues koennen zum Engpass werden. Miss Run Queue, Pool Wait und In-Flight Requests statt nur CPU-Prozent.

Thread	Vom Betriebssystem geplanter Ausfuehrungsstrom.
Task	Von einer Runtime geplante logische Arbeitseinheit.
Lock	Koordiniert exklusiven oder gemeinsamen Zugriff.
Context Switch	Wechsel der laufenden Arbeit; nicht kostenlos.

7. DNS, TCP, TLS

Bevor eine API antwortet, muss ein Pfad entstehen.

DNS uebersetzt Namen in erreichbare Ziele. Caches und TTLs machen diese Zuordnung schnell, aber nicht sofort aenderbar. TCP erzeugt einen geordneten Bytestrom und behandelt Verlust sowie Wiederuebertragung. TLS authentifiziert die Gegenstelle und verschluesselt den Transport.

Verbindungen kosten Handshakes und Zustand. Connection Pooling und Keep-Alive sparen diese Arbeit. Zu viele Verbindungen koennen dagegen Load Balancer, NAT Tables oder Backends erschoepfen. Grenzen gelten immer entlang der gesamten Kette.

Ein Netzwerk ist nicht nur Bandbreite. Round Trips, Paketverlust und Entfernung bestimmen Latenz. Multi-Region-Design kann die Lichtgeschwindigkeit nicht abstrahieren. Daten und Compute muessen nahe am kritischen User Flow liegen.

TTL	Wie lange DNS-Antworten typischerweise gecacht werden duerfen.
RTT	Zeit fuer Hin- und Rueckweg eines Netzwerkpakets.
Keep-Alive	Wiederverwendung einer bestehenden Verbindung.
mTLS	Beide Seiten authentifizieren sich mit Zertifikaten.

8. HTTP, Streaming und RPC

Waehle das Protokoll nach Interaktion, nicht nach Mode.

HTTP und JSON sind universell, debugbar und ideal fuer externe APIs. gRPC und Protobuf liefern kompakte, stark typisierte interne Vertrage und Streaming. WebSockets oder bidirektionale Streams passen zu lang laufender Interaktion, erzeugen aber dauerhaften Connection State.

AI-Antworten profitieren von Server Streaming: Der Nutzer sieht frueh Fortschritt, obwohl die Gesamtdauer gleich bleibt. Streaming erfordert Backpressure, Abbruchpropagation und saubere Semantik fuer teilweise Antworten.

Ein API-Vertrag ist ein Produkt. Verwende versionierte Schemas, additive Felder, stabile Fehlercodes, Idempotency Keys und Deadlines. Ein schneller Transport heilt keinen instabilen Vertrag.

MENTAL MODEL

```
external clients  -> HTTP + JSON + streaming
internal services -> typed RPC where it earns its cost
events            -> durable log or queue
large artifacts   -> object storage + signed references
```

9. Load Balancing und Routing

Verteile nicht nur Traffic; verteile Risiko.

Ein Load Balancer waehlt ein gesundes Ziel. Algorithmen reichen von Round Robin ueber Least Requests bis zu konsistentem Hashing. Health Checks muessen echte Bereitschaft messen, nicht nur einen lebenden Prozess.

Stateful Sessions machen Routing schwieriger. Sticky Sessions sind einfach, binden Nutzer aber an Instanzen und erschweren Failover. Besser ist externer oder explizit partitionierter State, wenn die Latenz es erlaubt.

AI-Routing beruecksichtigt mehr als CPU: geladenes Modell, freie GPU-Memory, KV-Cache-Lokalitaet, Tenant-Prioritaet, Context-Laenge und erwartete Output-Tokens. Ein Router kann Kosten und Cache Hit Rate stark beeinflussen.

L4	Routing anhand von Netzwerkverbindungen.
L7	Routing anhand von HTTP- oder Anwendungsdaten.
Affinity	Aehnliche Arbeit landet bewusst auf demselben Ziel.
Readiness	Instanz kann jetzt neue Arbeit korrekt annehmen.

10. Caches und CDNs

Ein Cache ist eine zweite Wahrheit mit Ablaufdatum.

Caching tauscht Frische und Komplexitaet gegen Latenz und Last. Definiere zuerst Cache Key, Gueltigkeit, Invalidierung und Verhalten bei Miss. Ohne diese vier Antworten ist ein Cache nur versteckter State.

Cache-aside liest zuerst Cache, dann Source of Truth und schreibt das Ergebnis zurueck. Write-through aktualisiert Cache im Write-Pfad. Jeder Ansatz kann Stampedes erzeugen, wenn viele Requests denselben fehlenden Key gleichzeitig laden.

CDNs bringen statische oder sicher gecachte Inhalte an den Edge. Private AI-Antworten sind selten direkt global cachebar. Prompt Prefixes, Embeddings und Retrieval-Zwischenergebnisse koennen dagegen unter klarer Tenant-Isolation grosse Einsparungen bringen.

TTL	Zeitbasierte Staleness-Grenze.
Stampede	Viele Misses laden gleichzeitig dieselbe Quelle.
Negative Cache	Auch ein Nichtfund wird kurz gespeichert.
Eviction	Welche Eintraege bei Platzdruck entfernt werden.

TEIL 3 / DATA SYSTEMS

Daten sind Entscheidungen ueber Zeit

Das Datenmodell bestimmt, welche Fragen billig sind, welche Konflikte moeglich werden und welche Wahrheit das System nach einem Crash noch kennt.

11. Datenmodell vor Datenbank

Modelliere Invarianten und Zugriffswege, nicht Tabellen aus Gewohnheit.

Relationale Modelle sind stark bei Beziehungen, Constraints und Transaktionen. Dokumentmodelle passen zu aggregierten Objekten mit flexibler Form. Key-Value Stores bieten einfache, schnelle Zugriffe. Graphen modellieren tiefe Beziehungen. Keine Kategorie ist universell besser.

Beginne mit Entities, Identitaeten, Ownership und Lebenszyklus. Welche Daten muessen gemeinsam atomar aenderbar sein? Welche Abfragen dominieren? Welche Daten wachsen unbegrenzt? Das logische Modell kommt vor der Wahl eines Produkts.

Denormalisierung kann Reads beschleunigen, erzeugt aber mehrere Kopien derselben Wahrheit. Jede Kopie braucht eine Update- und Repair-Strategie. Read Models duerfen eventual consistent sein, wenn das Produkt das explizit erlaubt.

Source of Truth	Autoritative Darstellung eines Fakts.
Projection	Fuer einen Read Path optimierte abgeleitete Sicht.
Constraint	Datenbankregel gegen ungueltige Zustaende.
Schema Evolution	Kompatible Veraenderung gespeicherter Formen.

12. Indizes und Query Plans

> Jeder Index ist eine Wette auf einen Zugriffsweg.

B-Trees bedienen Gleichheit, Sortierung und Bereiche. Invertierte Indizes verbinden Terme mit Dokumenten. LSM-Trees optimieren hohe Write-Raten durch sortierte Runs und Compaction. Vector-Indizes suchen geometrische Nachbarn.

Indizes kosten Speicher, Write Amplification und Cache. Ein Composite Index funktioniert nur fuer passende Praefixe und Sortierungen. Partial Indizes sparen Arbeit, wenn nur ein kleiner Zustand oft gelesen wird.

Der Query Planner entscheidet die physische Ausfuehrung. Schaetzfehler bei Cardinality koennen einen schlechten Join oder Scan waehlen. Nutze EXPLAIN mit realistischen Daten; ein SQL-Statement beweist seinen Plan nicht selbst.

Selectivity	Anteil der Zeilen, die ein Filter uebrig laesst.
Covering Index	Enthaelt alle fuer die Query benoetigten Daten.
Write Amplification	Ein logischer Write erzeugt mehrere physische Writes.
Compaction	Zusammenfuehren sortierter Datenstrukturen im Hintergrund.

13. Transaktionen und Isolation

ACID ist ein Vertrag ueber konkurrierende Zeit.

Atomicity macht eine Gruppe von Writes zu Alles-oder-nichts. Isolation beschreibt, welche Zwischenzustaende konkurrierende Transaktionen sehen. Durability beschreibt, was nach bestaetigtem Commit einen Crash ueberlebt.

READ COMMITTED verhindert Dirty Reads, aber derselbe Ablauf kann spaeter neue bestaetigte Daten sehen. REPEATABLE READ stabilisiert den Snapshot. SERIALIZABLE versucht, das Ergebnis einer seriellen Reihenfolge gleichzusetzen und kann Transaktionen zum Retry zwingen.

Isolation ersetzt nicht das Domain-Modell. Eindeutige Constraints, Compare-and-Set, Row Locks oder fachliche Advisory Locks machen konkrete Konflikte sichtbar. Halte Transaktionen kurz; Netzwerkaufrufe gehoeren nicht in offene DB-Transaktionen.

Lost Update	Konkurrierende Writer ueberschreiben einander.
Write Skew	Einzeln gueltige Entscheidungen verletzen gemeinsam eine Regel.
Deadlock	Transaktionen warten zyklisch; eine wird abgebrochen.
Retry	Serializable Fehler sind Teil des normalen Kontrollflusses.

14. Replication

> Replikation macht Kopien, nicht automatisch Verfuegbarkeit.

Leader-Follower-Replikation ordnet Writes an einem Leader und repliziert sie. Reads von Followern skalieren, koennen aber hinterherhinken. Multi-Leader erlaubt regionale Writes, verschiebt Konflikte aber in die Anwendung.

Synchrone Replikation verbessert Durability, erhoeht jedoch Write-Latenz und kann bei Partitionen Verfuegbarkeit reduzieren. Asynchrone Replikation ist schneller, riskiert aber bei Failover bestaetigte letzte Writes.

Definiere Read-your-writes und monotone Reads dort, wo Nutzer sie erwarten. Routing kann denselben Nutzer kurz an den Leader oder eine ausreichend aktuelle Replica binden. Konsistenz ist eine User-Experience-Eigenschaft.

MENTAL MODEL

```
client write -> leader -> replication log -> followers
                  |             |
              acknowledged    possible lag
```

15. Partitioning und Hot Keys

Sharding verteilt Daten; der Partition Key verteilt Last.

Hash Partitioning verteilt gleichmaessig, erschwert aber Range Scans. Range Partitioning macht Zeit- oder Wertebereiche effizient, kann aber Hot Partitions erzeugen. Tenant-basierte Partitionierung verbessert Isolation, wenn Tenant-Groessen kontrolliert werden.

Ein Hot Key kann einen ganzen Cluster limitieren. Beruehmte Nutzer, globale Counter oder identische Prompt Prefixes erzeugen unausgewogene Last. Salting, lokale Aggregation, Hierarchien oder dedizierte Shards helfen, veraendern aber Semantik.

Rebalancing ist ein verteilter Migrationsprozess. Daten werden kopiert, Writes koordiniert, Checksummen verglichen, Routing umgestellt und alte Kopien entfernt. Plane ihn vor dem ersten echten Shard.

Partition Key	Bestimmt Ort und Lastverteilung eines Records.
Hotspot	Ueberproportionale Last auf wenigen Partitionen.
Rebalance	Verschieben von Ownership zwischen Shards.
Tenant Placement	Bewusste Zuordnung eines Kunden zu Cell oder Shard.

16. Consistency als Produktentscheidung

Strong und eventual sind keine moralischen Kategorien.

Starke Konsistenz ist wertvoll fuer Geld, Berechtigungen, Inventargrenzen und Konfliktentscheidungen. Eventual Consistency passt zu Suche, Analytics, Embeddings und viele Read Models. Die richtige Frage lautet: Welche falsche Beobachtung kann ein Nutzer machen und wie lange?

CAP beschreibt Entscheidungen waehrend einer Netzwerkpartition. Es bedeutet nicht, dass ein Produkt beliebig zwei Buchstaben waehlt. In realen Systemen bestimmen Latenz, Failure Detection, Quorums und Datenart das Verhalten.

CRDTs loesen bestimmte zusammenfuehrbare Datentypen ohne zentrale Reihenfolge. Sie machen nicht jede Business-Regel konfliktfrei. Wenn zwei Agents denselben exklusiven Auftrag annehmen, braucht die Domain weiterhin eine Entscheidung.

Linearizable	Operationen wirken wie eine einzige Echtzeit-Reihenfolge.
Eventual	Kopien konvergieren, wenn keine neuen Writes eintreffen.
Causal	Ursache-Wirkung-Reihenfolgen bleiben sichtbar.
Conflict Policy	Explizite Regel, welche konkurrierende Version gewinnt.

17. Queues, Logs und Streams

Eine Queue verteilt Arbeit; ein Log bewahrt Reihenfolge.

Work Queues geben jede Aufgabe typischerweise an einen Consumer. Durable Logs behalten Events und lassen mehrere Consumer-Gruppen unabhaengig lesen. Streams sind Logs plus laufende Transformationen und Zustandsbildung.

At-least-once Delivery erzeugt moegliche Duplikate. Consumer brauchen Idempotenz. Ordering gilt meist nur pro Partition; der Partition Key entscheidet deshalb auch semantische Reihenfolge.

Consumer Lag ist gespeicherte Wartezeit. Miss nicht nur Anzahl Nachrichten, sondern Alter des aeltesten Events, Durchsatz, Retry Rate und Poison Items. Eine Queue ohne Backpressure ist ein langsamer Speicherunfall.

Offset	Position eines Consumers im Log.
Consumer Group	Teilt Partitionen einer Arbeitsgruppe zu.
DLQ	Quarantaene fuer wiederholt nicht verarbeitbare Items.
Replay	Erneutes Lesen alter Events fuer Repair oder neue Projections.

18. Batch und Stream zusammen denken

> Batch ist ein Stream mit spaeter Deadline; Stream ist inkrementeller Batch.

Batch Processing maximiert Effizienz, wenn Minuten oder Stunden Latenz akzeptabel sind. Streaming minimiert Freshness, kostet aber mehr Zustands-, Ordering- und Betriebslogik. Viele Systeme brauchen beides.

Ein robustes Muster bewahrt rohe unveraenderliche Events, erzeugt schnelle inkrementelle Views und kann sie spaeter aus der Quelle neu bauen. Recompute ist ein Repair-Mechanismus und erfordert deterministische Versionen.

AI-Pipelines nutzen Batch fuer Embeddings, Training, grosse Evals und Backfills; Streaming fuer Online Features, Agent Events und aktuelle Indizes. Frischeziele sollten je Datenprodukt definiert werden.

MENTAL MODEL

```
raw immutable data
      |-- streaming -> fresh serving view
      |-- batch ----> exact backfill / repair / training
```

TEIL 4 / CLASSIC SYSTEM DESIGN CORE

Die Grundlagen unter jeder modernen Plattform

PostgreSQL, Auth, globale IDs, Consistent Hashing, Consensus und CAP bleiben zentral - auch wenn das Produkt Models und Agents benutzt.

18A. PostgreSQL in Produktion

> PostgreSQL ist oft die beste erste Source of Truth - wenn man seine Betriebsmechanik versteht.

PostgreSQL verwendet MVCC: Updates erzeugen neue Row-Versionen, waehrend laufende Transaktionen alte Snapshots weiter lesen koennen. Das reduziert Read-Write-Blocking, erzeugt aber tote Tupel. Autovacuum raeumt sie auf und verhindert, dass Transaction IDs unbrauchbar werden.

Das Write-Ahead Log wird vor den eigentlichen Datenseiten dauerhaft geschrieben. Es ermoeglicht Crash Recovery, Point-in-Time Restore und Streaming Replication. Checkpoints begrenzen Recovery-Arbeit, koennen aber I/O-Spitzen erzeugen. Durability ist eine Kette aus Anwendung, WAL, Storage und Backup.

Connection Pools sind Pflicht, weil jede Backend-Verbindung serverseitigen Zustand und Speicher kostet. Indizes beschleunigen passende Queries, erhoehen aber Write Amplification und Bloat. EXPLAIN ANALYZE, Lock Waits, Pool Wait, Replica Lag und Vacuum Health gehoeren zur normalen Observability.

MVCC	Mehrere Row-Versionen erlauben konsistente Snapshots.
WAL	Sequentielles Log fuer Durability, Recovery und Replikation.
Autovacuum	Entfernt tote Versionen und pflegt Transaktionsgrenzen.
PgBouncer	Multiplexiert viele Clients auf kontrollierte DB-Verbindungen.
Replica Lag	Zeit oder Log-Distanz zwischen Primary und Follower.

18B. Authentication und Authorization

AuthN beweist Identitaet; AuthZ entscheidet konkrete Befugnis.

Authentication beantwortet: Wer ist der Actor? Authorization beantwortet: Darf dieser Actor diese Aktion auf genau dieser Ressource ausfuehren? Beide Pruefungen muessen serverseitig stattfinden. Client-Felder wie role=admin sind untrusted Input.

Browser-Sessions speichern eine zufaellige Session-ID im sicheren Cookie und halten autoritativen State serverseitig. JWTs tragen signierte Claims und koennen Services ohne Session-Lookup pruefen, erschweren aber sofortige Revocation und Claim-Aenderungen. Kurze Laufzeit, Audience, Issuer, Algorithmus und Key Rotation muessen geprueft werden.

OAuth 2.0 ist delegierte Autorisierung; OpenID Connect ergaenzt Identitaet. API Keys identifizieren Maschinen oder Agents und muessen gehasht, scoped, rotierbar und widerrufbar sein. RBAC ordnet Rollen Rechte zu; ABAC und Policy Engines beruecksichtigen Ressource, Tenant, Kontext und Aktion.

Session	Opaque ID; autoritativer Zustand bleibt serverseitig.
JWT	Signierter Claim-Container mit Ablauf und Zielgruppe.
OAuth	Delegierte Autorisierung fuer einen Client.
OIDC	Identity Layer auf OAuth 2.0.
RBAC / ABAC	Rollenbasierte oder attributbasierte Policy.

18C. Globale IDs und Zeit

Eindeutigkeit und Reihenfolge sind verschiedene Eigenschaften.

Auto-Increment IDs sind innerhalb einer Datenbank einfach, brauchen aber eine zentrale Sequenz. UUIDv4 ist dezentral und zufaellig. UUIDv7 und ULID sind weitgehend zeitlich sortierbar und indexfreundlicher. Snowflake-artige IDs kombinieren Zeit, Worker-ID und Sequenz, verlangen aber kontrollierte Clocks und Worker-Zuordnung.

Wall Clocks koennen durch NTP springen. Verwende monotone Uhren fuer Dauern und Deadlines. Verteilte Events besitzen selten eine perfekte globale Reihenfolge. Wenn Reihenfolge fachlich wichtig ist, braucht sie einen Log-Offset, eine Version oder einen Single Writer pro Schluessel.

IDs duerfen keine geheimen Informationen oder leicht erratbare Autoritaet tragen. Eine ID findet eine Ressource; Authorization entscheidet Zugriff. Idempotency Keys identifizieren eine Operation und sind nicht dasselbe wie Record IDs.

UUIDv4	Zufaellige dezentrale 128-Bit-ID.
UUIDv7	Zeitlich sortierbare UUID fuer verteilte Systeme.
Snowflake	Zeit plus Worker plus lokale Sequenz.
Monotonic Clock	Geeignet fuer Dauer; springt nicht mit Wall Time.

18D. Consistent Hashing und Shard Maps

Datenverteilung braucht stabile Ownership bei veraenderlicher Flotte.

Normales hash(key) modulo N verschiebt fast alle Keys, wenn N sich aendert. Consistent Hashing legt Nodes und Keys auf einen Ring; beim Hinzufuegen oder Entfernen wandert nur ein Teil. Virtual Nodes verbessern Verteilung bei ungleichen physischen Nodes.

Rendezvous Hashing waehlt fuer einen Key die hoechst bewerteten Nodes und ist oft einfacher fuer replizierte Zuordnung. Ein explizites Shard Directory bietet mehr Kontrolle fuer Tenant Placement und Hot-Tenant-Migration, wird aber selbst zu kritischem State.

Hashing allein loest keine Replikation, Rebalancing oder Hot Keys. Der Partition Key muss fachliche Zugriffe und Last reflektieren. Bewahre Versionen der Ownership Map und leite Requests waehrend Migration deterministisch weiter.

Hash Ring	Kreisfoermiger Keyspace mit Node-Positionen.
Virtual Node	Mehrere Positionen pro physischem Node.
Rendezvous	Waehlt Nodes ueber deterministische Scores.
Shard Map	Explizite Zuordnung von Range oder Tenant zu Shard.

18E. Consensus, Leader Election und Locks

Ein verteilter Lock ist ein Protokoll, kein Boolean in Redis.

Consensus-Protokolle wie Raft oder Paxos lassen mehrere Nodes eine gemeinsame Log-Reihenfolge oder Leader-Entscheidung trotz bestimmter Ausfaelle vereinbaren. Sie benoetigen Quorums und koennen bei fehlender Mehrheit keinen sicheren Fortschritt machen.

Leader Election bestimmt einen Actor fuer exklusive Arbeit. Leases laufen zeitlich ab, aber ein pausierter alter Leader kann spaeter weiterarbeiten. Fencing Tokens sind monoton steigende Versionen, die der geschuetzte Speicher prueft und alte Writer ablehnt.

Nutze Datenbank-Constraints, Compare-and-Set oder eine transaktionale Single-Writer-Grenze, bevor du einen externen Distributed Lock einfuehrst. Locks ohne Ownership-Verifikation, Ablauf und Fencing erzeugen scheinbare Sicherheit.

Quorum	Mehrheit oder definierte Teilmenge fuer eine sichere Entscheidung.
Term	Epoche einer Leader-Amtszeit.
Lease	Zeitlich begrenzte Ownership.
Fencing Token	Monotone Version; alte Owner werden am Ziel abgewiesen.

18F. CAP, PACELC und Quorums

Partitionen erzwingen Entscheidungen; auch ohne Partition bezahlen wir mit Latenz.

CAP sagt: Wenn das Netzwerk zwischen Replikaten partitioniert ist, kann ein System nicht gleichzeitig jede Anfrage verfuegbar beantworten und eine einzige linearisierbare Wahrheit garantieren. Consistency bedeutet hier einen spezifischen starken Begriff, nicht allgemein gute Daten.

PACELC erweitert das Denken: Bei einer Partition waehlen wir Availability oder Consistency; Else, im Normalbetrieb, waehlen wir oft niedrigere Latency oder staerkere Consistency. Synchrone Quorums erhoehen Sicherheit und Latenz.

Bei N Replikaten kann R plus W groesser als N ueberlappende Read- und Write-Quorums erzeugen, aber nur mit korrekter Versionierung, Repair und Failure Semantik. Quorum-Formeln allein garantieren keine linearisierbare Anwendung.

Partition	Nachrichten zwischen Teilen des Systems kommen nicht verlaesslich an.
Availability	Jede nicht ausgefallene Seite antwortet innerhalb einer Grenze.
Linearizability	Operationen wirken wie eine Echtzeit-Reihenfolge.
PACELC	Partition: A/C; Else: Latency/Consistency.

18G. API Gateway und Service Security

Die Edge erzwingt gemeinsame Grenzen; Domain-Services besitzen fachliche Policy.

Ein API Gateway kann TLS beenden, Routing, Auth-Pruefung, Request IDs, Rate Limits, WAF-Regeln und grobe Quoten durchsetzen. Es sollte nicht zum zentralen Ort jeder Business-Regel werden. Fachliche Authorization bleibt nahe an Ressource und Source of Truth.

Service-to-Service-Kommunikation braucht eindeutige Workload Identity, kurze Credentials und Least Privilege. mTLS authentifiziert beide Verbindungsseiten; ein Service Mesh kann Zertifikate und Telemetrie automatisieren, erhoeht aber Control-Plane- und Debugging-Komplexitaet.

Secrets liegen in einem Secret Manager, werden rotiert und niemals in Images, Logs oder Prompts gespeichert. Egress Policies begrenzen, welche Ziele ein kompromittierter Service erreichen kann. Zero Trust bedeutet kontinuierlich pruefbare Identitaet und Policy, nicht Misstrauen als Marketingwort.

Gateway	Gemeinsame Edge-Kontrollen und Routing.
Workload Identity	Kryptografisch pruefbare Dienstidentitaet.
mTLS	Gegenseitige Zertifikatsauthentifizierung.
Secret Rotation	Credentials wechseln ohne langen Ausfall.

18H. PostgreSQL first, Polyglot spaeter

Eine gute relationale Datenbank plus Object Storage traegt erstaunlich weit.

PostgreSQL kann Transaktionen, JSON, Full-Text, Extensions, Replicas und mit pgvector auch viele fruehe Vector-Workloads bedienen. Object Storage bewahrt grosse unveraenderliche Artefakte. Redis oder ein Managed Cache beschleunigt temporaere Reads, darf aber nicht unbemerkt zur einzigen Wahrheit werden.

Spezialisierte Search-, Time-Series-, Graph- oder Vector-Systeme werden sinnvoll, wenn ein gemessener Workload ihre Staerken braucht. Jede weitere Datenbank erzeugt Schema-Evolution, Backfill, Delete Propagation, Monitoring, Backup und On-Call-Arbeit.

Polyglot Persistence ist ein Ergebnis unterschiedlicher Zugriffswege, kein Architekturziel. Definiere fuer jede Kopie Source of Truth, Freshness SLO und Repair-Pfad. Sonst entstehen widerspruechliche Dateninseln.

PostgreSQL	Autoritative relationale Daten und Transaktionen.
Object Storage	Grosse dauerhafte Blobs und Rohdaten.
Cache	Ephemere, ableitbare Beschleunigung.
Search / Vector	Spezialisierte abgeleitete Serving Indizes.

TEIL 5 / CLOUD INFRASTRUCTURE

Cloud ist ein Failure- und Ownership-Modell

Managed Services entfernen Arbeit, aber nicht Verantwortung. Regionen, Zones, Accounts, Deployments und Quoten formen die reale Architektur.

19. Regionen und Availability Zones

Entfernung, Isolation und Datenhoheit sind physische Fakten.

Eine Region ist ein geografischer Bereich; Availability Zones sollen unabhaengige Failure Domains innerhalb einer Region bilden. Multi-AZ schuetzt gegen lokale Ausfaelle. Multi-Region adressiert groessere Katastrophen, globale Latenz oder Datenresidenz - mit deutlich mehr Konsistenzkomplexitaet.

Verteile Replikate nicht nur, sondern pruefe gemeinsame Abhaengigkeiten: Control Plane, Secrets, DNS, Identity Provider und Deployment Pipeline. Scheinbar getrennte Services koennen denselben Blast Radius teilen.

Datenresidenz ist Teil des Routings. Ein Tenant sollte eine Home Region besitzen. Globale Metadaten bleiben klein; der normale Datenpfad bleibt regional. Failover braucht eine klare Autoritaetsuebergabe statt gleichzeitiger Writer.

Failure Domain	Bereich, der durch eine gemeinsame Ursache ausfallen kann.
Home Region	Primaerer Ort fuer Tenant Writes und autoritativen State.
RPO	Maximal akzeptierter Datenverlust in Zeit.
RTO	Maximal akzeptierte Wiederherstellungsdauer.

20. VMs, Container und Bare Metal

Waehle die Isolationseinheit nach Workload und Betrieb.

VMs virtualisieren Hardware und liefern starke Isolation. Container teilen den Host-Kernel und starten schneller. Bare Metal maximiert Kontrolle und kann fuer grosse Accelerator-Flotten attraktiv sein, erhoeht aber Betriebsarbeit.

Stateless API Compute passt gut zu replizierten Containern. Datenbanken und Queues brauchen dauerhaften State und kontrollierte Upgrades. GPU Serving benoetigt Device Plugins, Treiberkompatibilitaet, Topologiekenntnis und oft laengere Warmup-Zeiten.

Die richtige Abstraktion ist die niedrigste, die das Team zuverlaessig betreiben kann. Managed Model APIs, Managed Kubernetes oder eigene GPU-Cluster sind verschiedene Ownership-Entscheidungen, keine Reifestufen.

Image	Unveraenderliches Paket fuer Code und Laufzeit.
Node	Maschine, auf der Workloads geplant werden.
Accelerator	GPU, TPU oder spezialisiertes Inference-Silicon.
Warmup	Zeit bis Modell, Cache und Runtime wirklich bereit sind.

21. Kubernetes als Control Loop

> Kubernetes ist ein Reconciler, kein Auto-Architect.

Du beschreibst Desired State; Controller beobachten Actual State und versuchen die Differenz zu schliessen. Deployments verwalten stateless Replikate, StatefulSets stabile Identitaeten, Services Discovery und Routing, Jobs endliche Arbeit.

Readiness entscheidet, ob ein Pod Traffic erhalten darf. Liveness entscheidet, ob ein Neustart helfen kann. Startup Probes schuetzen langsame Initialisierung. Eine falsche Liveness Probe kann eine ueberlastete Anwendung durch Neustartschleifen zerstoeren.

Kubernetes loest nicht Datenkonsistenz, API-Vertraege oder Capacity Planning. Es macht Deployment und Scheduling programmierbar. Nutze es, wenn genug Workloads und Teamfaehigkeit seine Control Plane rechtfertigen.

MENTAL MODEL

```
desired state -> controller -> actual state
      ^                          |
      +---------- observe -------+
```

22. Autoscaling und das falsche Signal

Scale auf die Warteschlange, die Nutzer spueren.

Horizontal Autoscaling erhoeht Replikate; vertical vergroessert Ressourcen; Cluster Autoscaling fuegt Nodes hinzu. Jede Stufe hat andere Reaktionszeit. GPU-Nodes koennen Minuten brauchen, waehrend Traffic in Sekunden steigt.

CPU ist fuer API Compute oft brauchbar, fuer LLM Serving aber unvollstaendig. Queue Depth, Tokens pro Sekunde, TTFT, KV-Cache-Druck und laufende Sequences sind naehere Signale. Verwende Stabilization Windows und begrenzte Scale-down-Raten gegen Flapping.

Autoscaling ersetzt keine Admission Control. Wenn neue Kapazitaet zu spaet kommt, muss das System Arbeit priorisieren, begrenzen oder ablehnen. Sonst waechst die Queue schneller als die Flotte.

HPA	Skaliert Workload-Replikate anhand beobachteter Metriken.
VPA	Passt Ressourcenanforderungen einzelner Workloads an.
Cluster Autoscaler	Aendert die Anzahl verfuegbarer Nodes.
Predictive	Plant Kapazitaet vor bekannten Lastmustern.

23. Infrastructure as Code und Deployments

> Reproduzierbarkeit ist eine Zuverlaessigkeitseigenschaft.

Infrastructure as Code beschreibt Netzwerke, Rollen, Datenbanken und Cluster versioniert. Der Plan muss reviewbar sein. Secrets gehoeren in einen Secret Store, nicht in State Files oder Repositories.

Deployments sollten klein, beobachtbar und reversibel sein. Rolling Updates sparen Kapazitaet, Canaries begrenzen Risiko, Blue-Green vereinfacht Umschalten. Datenbankschemas brauchen Expand-and-Contract, weil alte und neue Versionen zeitweise gleichzeitig laufen.

Ein Rollback von Code macht eine destruktive Migration nicht rueckgaengig. Trenne Schema-Evolution, Daten-Backfill und Verhaltensumschaltung. Automatisiere Verifikation, nicht nur Ausfuehrung.

Canary	Kleine Traffic-Menge prueft eine neue Version.
Blue-Green	Zwei Umgebungen, Routing wird kontrolliert umgestellt.
Feature Flag	Verhalten ohne neues Deployment aktivieren oder stoppen.
Drift	Reale Infrastruktur weicht von deklarierter Konfiguration ab.

24. Serverless und Managed Services

Bezahle mit Kontrolle, um Betriebsarbeit zu entfernen.

Serverless Compute passt zu bursty, kurzen und weitgehend stateless Workloads. Cold Starts, Laufzeitgrenzen, Egress und Provider-Limits koennen unpassend sein. AI Inference mit grossen Modellen besitzt wertvolle warme GPU- und KV-Cache-Zustaende.

Managed Datenbanken, Queues und Model APIs reduzieren On-Call- und Upgrade-Arbeit. Sie entfernen aber nicht Datenmodell, Capacity Limits, Kosten, Quoten oder Vendor Failure. Lies Limits und Exportpfade vor dem Wachstum.

Build versus Buy ist eine Oekonomiefrage: Differenziert diese Komponente das Produkt? Ist der Workload stabil genug fuer eigene Optimierung? Hat das Team die operative Tiefe? Opportunity Cost ist oft groesser als Cloud-Aufschlag.

Cold Start	Initialisierung vor der ersten Verarbeitung.
Egress	Kosten und Latenz fuer Daten aus einer Provider-Grenze.
Quota	Vom Provider gesetzte Kapazitaetsgrenze.
Lock-in	Kosten eines spaeteren Wechsels von API, Daten oder Betrieb.

25. Cells und Shuffle Sharding

> Globales Wachstum braucht kleine, bekannte Failure Units.

Eine Cell ist eine weitgehend selbststaendige Kopie des Serving Stacks fuer eine begrenzte Tenant-Gruppe. Ein duennes Directory routet Tenants. Neue Cells erweitern Kapazitaet; ein Fehler bleibt auf einen bekannten Anteil begrenzt.

Shuffle Sharding gibt Kunden kleine, teilweise ueberlappende Teilmengen einer Flotte. Dadurch teilen wenige Kunden exakt denselben Failure Set. Es ist besonders stark fuer stateless oder replizierte Serving Pools.

Cells sind kein kostenloses Sharding. Placement, Migration, globale Konfiguration, Deployment Wellen und Cell Observability werden zu eigenen Systemen. Beginne erst, wenn ein einzelner Stack als Capacity Unit verstanden und automatisiert ist.

MENTAL MODEL

```
global directory
  |-- tenant A -> cell 03 -> self-contained stack
  |-- tenant B -> cell 11 -> self-contained stack
  |-- tenant C -> cell 03 -> bounded shared fate
```

TEIL 6 / RELIABILITY

Das System unter Druck

Zuverlaessigkeit entsteht aus Grenzen: Deadlines, Idempotenz, Backpressure, Isolation, SLOs und getesteter Wiederherstellung.

26. Timeouts, Retries und Idempotenz

> Ein Retry ist ein neuer Request mit alter Absicht.

Timeouts begrenzen Ressourcen und Nutzerwartezeit. Retries helfen bei transienten Fehlern, koennen einen ueberlasteten Dienst aber weiter treffen. Verwende exponential Backoff, Jitter, Retry Budgets und nur wenige Versuche innerhalb der Deadline.

Eine Mutation ist nur sicher wiederholbar, wenn sie idempotent ist. Der Client sendet eine stabile Operation ID; die Datenbank erzwingt Eindeutigkeit; der Server gibt bei Wiederholung dasselbe Ergebnis zurueck.

Hedged Requests koennen langsame Tails reduzieren, verdoppeln aber Last. Sie passen nur zu idempotenten Reads, nach kurzer Verzoegerung und mit globalem Budget. Reliability Patterns sind ohne Kapazitaetsmodell gefaehrlich.

Backoff	Wachsende Pause zwischen Versuchen.
Jitter	Zufaellige Streuung gegen synchronisierte Retry-Wellen.
Idempotency Key	Stabile Identitaet einer fachlichen Mutation.
Retry Budget	Maximaler Zusatztraffic durch Wiederholungen.

27. Backpressure und Overload

> Wenn alles wichtig ist, faellt alles gemeinsam aus.

Backpressure signalisiert Produzenten, dass Verbraucher nicht nachkommen. Bounded Queues machen Kapazitaet sichtbar. Unbounded Queues verwandeln Ueberlast in hohe Latenz, Memory-Druck und spaeten Totalausfall.

Admission Control lehnt neue Arbeit ab, bevor sie knappe Ressourcen verbraucht. Prioritaeten schuetzen interaktive Nutzer vor Batch Jobs. Load Shedding entfernt Arbeit mit geringstem Wert. Gute Systeme degradieren bewusst.

Bei AI ist Output-Laenge ein unbekannter Ressourcenverbrauch. Reserviere konservativ, setze Token-Limits, stoppe verlorene Streams und trenne lange von kurzen Requests. Sonst blockiert ein kleiner Anteil grosse Kontexte die gesamte Flotte.

MENTAL MODEL

```
incoming work -> admission -> bounded queue -> workers
                    |              |
                 reject       deadline expiry
```

28. Rate Limits und Circuit Breakers

Schuetze gemeinsame Kapazitaet vor einzelnen Fehlern.

Token Buckets erlauben Bursts und begrenzen dauerhafte Rate. Limits koennen pro User, Tenant, API Key, Region oder Kostenbudget gelten. Eine globale Grenze gehoert an eine geteilte oder deterministisch partitionierte Stelle; lokale Limits bleiben als Notbremse.

Circuit Breakers stoppen Aufrufe zu einer bereits fehlerhaften Abhaengigkeit und erlauben spaeter Probe-Requests. Bulkheads trennen Ressourcenpools, damit ein Workload nicht alle Threads, Connections oder GPUs belegt.

Rate Limits sind Produktpolitik. 429 sollte Retry-Information und ein verstaendliches Budgetmodell liefern. Fuer Agents sind Token-, Tool- und Geldbudgets oft wichtiger als reine Requests pro Sekunde.

Token Bucket	Rate plus erlaubte Burst-Kapazitaet.
Circuit Open	Aufrufe werden sofort abgelehnt oder fallen zurueck.
Bulkhead	Getrennter Ressourcenpool fuer Failure Isolation.
Fair Queueing	Kapazitaet wird zwischen Tenants gerecht verteilt.

29. SLOs und Error Budgets

Reliability braucht eine Zahl und einen User Flow.

Ein SLI misst Nutzererfahrung, etwa erfolgreiche Context Builds unter einer Latenzgrenze. Ein SLO setzt das Ziel. Das Error Budget ist der erlaubte Anteil schlechter Ereignisse. Es verbindet Produktgeschwindigkeit und Zuverlaessigkeit.

Messe nicht nur Service Uptime. Ein Endpoint kann 200 liefern und fachlich unbrauchbar sein. AI-SLIs koennen TTFT, Completion Rate, Tool Success, Groundedness oder Kosten pro erfolgreicher Aufgabe enthalten.

SLOs brauchen ein Fenster und klare Ausschluesse. Wenn das Budget verbraucht ist, werden riskante Releases reduziert und Reliability-Arbeit priorisiert. Sonst bleiben SLOs Dekoration.

SLI	Gemessener Indikator eines User Outcomes.
SLO	Ziel fuer den Indikator in einem Zeitraum.
Error Budget	Erlaubte schlechte Ereignisse bis zum SLO-Verstoss.
Burn Rate	Wie schnell das Budget aktuell verbraucht wird.

30. Observability ohne Datenexplosion

Telemetry muss Fragen beantworten, nicht alles speichern.

Metrics zeigen aggregierte Zustaende, Logs diskrete Ereignisse, Traces den Weg eines Requests und Profiles den ressourcenverbrauchenden Code. Correlation IDs verbinden die Signale.

High-Cardinality Labels wie user_id oder prompt erzeugen unkontrollierte Zeitreihen und Kosten. Verwende stabile Dimensionen fuer Metrics, Details in gesampelten Traces und sichere strukturierte Logs ohne Secrets.

AI-Telemetry braucht Model, Version, Region, Queue-Zeit, Input- und Output-Tokens, Cache Hits, Retrieval Scores, Tool Outcomes und Kosten. Prompt-Inhalte sind sensible Nutzerdaten und gehoeren nicht automatisch in Logs.

Metrics	Was passiert aggregiert?
Logs	Welches diskrete Ereignis trat ein?
Traces	Wo verbrachte ein Request seine Zeit?
Profiles	Welcher Code verbrauchte CPU oder Memory?

31. Backup, Restore und Disaster Recovery

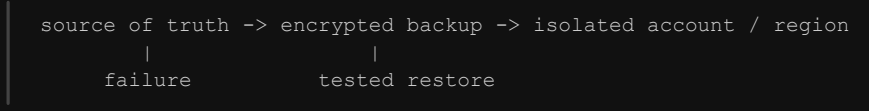
> Ein Backup ist erst real, wenn Restore gemessen wurde.

Definiere RPO und RTO pro Datenart. Audit, Zahlungen und Identitaet koennen strengere Ziele als abgeleitete Embeddings haben. Replika ist kein Backup: Fehlerhafte Writes und Loeschungen werden repliziert.

Backups brauchen Verschluesselung, getrennte Credentials, Retention und regelmaessige Restore-Uebungen. Ein Point-in-Time Restore muss mit Event Logs, Object Storage und Suchindizes wieder in einen konsistenten Zustand gebracht werden.

Disaster Recovery ist ein Runbook mit Autoritaet: Wer entscheidet Failover? Wie wird Split Brain verhindert? Wie wird zurueckmigriert? Automatisiere sichere Schritte, aber lasse irreversible Entscheidungen explizit.

MENTAL MODEL



TEIL 7 / AI INFRASTRUCTURE

Models veraendern die Oekonomie des Request-Pfads

LLM Serving ist Queueing, Memory Management und Scheduling auf teurer Accelerator-Hardware. Tokens sind die neue Arbeitseinheit.

32. Warum AI-Workloads anders sind

Der Code ist oft billig; Modellzustand und Tokens sind teuer.

Ein klassischer API-Request kostet oft Millisekunden CPU und wenige Kilobyte State. Ein LLM-Request kann Sekunden laufen, grosse GPU-Memory belegen, einen langen Stream offen halten und variable Output-Laenge erzeugen.

Qualitaet ist probabilistisch. Ein technisch erfolgreicher 200-Response kann fachlich falsch sein. Deshalb gehoeren Evals, Guardrails, Versionierung und Feedback in die Architektur, nicht nur Unit Tests.

AI-Systeme besitzen mindestens zwei Control Loops: Infrastruktur optimiert Queue, Latenz und Kosten; Produkt-Evaluation optimiert Qualitaet und Sicherheit. Beide muessen dieselbe Model- und Prompt-Version identifizieren koennen.

Token	Kleinste praktische Workload-Einheit fuer LLM-Kosten und Durchsatz.
Context	Eingabetokens, die Attention und KV-Cache beeinflussen.
Sampling	Probabilistische Wahl des naechsten Tokens.
Eval	Messbare Qualitaetspruefung auf reprasentativen Aufgaben.

33. Model Lifecycle

Ein Modell ist ein versioniertes Produktionsartefakt mit Datenabhaengigkeiten.

Der Lifecycle umfasst Auswahl oder Training, Evaluation, Packaging, Deployment, Shadow Traffic, Canary, Monitoring und Rollback. Prompt, Tool Schema, Retrieval und Policy sind Teil der effektiven Model-Version.

Ein Model Registry Record sollte Gewichte, Tokenizer, Quantisierung, Runtime, Eval-Ergebnisse, Datenherkunft und Freigabestatus verbinden. Reproduzierbarkeit ist fuer Incident Analyse und Compliance entscheidend.

Rollbacks sind nicht immer symmetrisch. Neue Embeddings, Cache-Formate oder Tool-Vertraege koennen mit alten Modellen inkompatibel sein. Verwende Compatibility Gates und parallele Versionen.

MENTAL MODEL

```
candidate -> offline eval -> shadow -> canary -> production
                  |            |          |
                reject       compare    monitor + rollback
```

34. GPU Memory und Compute

Accelerator-Auslastung ist ein Scheduling-Problem.

GPU-Memory enthaelt Modellgewichte, Runtime Buffers und KV-Cache. Passt ein Modell nicht auf ein Device, braucht es Quantisierung oder Verteilung. Hohe Compute-Auslastung ist gut, solange Queue und Tail Latency innerhalb des SLO bleiben.

HBM-Bandbreite, Rechenleistung und Interconnect bestimmen verschiedene Phasen. Grosse Modelle koennen memory-bound sein. Multi-GPU-Kommunikation ueber NVLink oder Netzwerk wird bei Parallelism zum kritischen Pfad.

Plane nicht nur durchschnittliche Utilization. Fragmentierung, lange Sequences und gemischte Context-Laengen koennen freie Memory unbrauchbar verteilen. Paged KV-Management reduziert diese Verschwendung.

Weights	Modellparameter, meist grosster statischer Speicherblock.
KV Cache	Attention-Zustand pro aktiver Sequence.
HBM	Schneller Accelerator-Speicher mit begrenzter Kapazitaet.
Interconnect	Verbindung zwischen Accelerators fuer verteilte Inference.

35. Prefill, Decode und KV Cache

Ein LLM-Request hat zwei verschiedene Leistungsphasen.

Prefill verarbeitet den gesamten Input parallel und erzeugt den initialen KV-Cache. Es ist oft compute-intensiv und wird durch lange Kontexte teuer. Decode erzeugt Token fuer Token und liest den wachsenden KV-Cache; es ist haeufig memory-bandwidth- und scheduling-sensitiv.

Time to First Token wird stark durch Queue plus Prefill bestimmt. Inter-Token-Latenz entsteht im Decode. Beide Workloads koennen getrennt skaliert oder disaggregiert werden, wenn die Netzwerk- und Cache-Transferkosten es rechtfertigen.

KV-Cache ist wertvoller lokaler State. Prefix Caching kann gemeinsame System Prompts oder Dokumentpraefixe wiederverwenden. Routing nach Cache-Lokalitaet spart Compute, muss aber Tenant-Isolation und Retention beachten.

MENTAL MODEL

```
prompt tokens -> PREFILL -> first token -> DECODE -> streamed tokens
                    |                        |
                 KV cache --------------------+
```

36. Continuous Batching und Queueing

Batching tauscht wenige Millisekunden Warten gegen viel mehr Durchsatz.

Statisches Batching wartet auf eine feste Gruppe. Continuous Batching fuegt neue Sequences hinzu, wenn andere fertig werden. Dadurch bleibt die GPU besser ausgelastet, obwohl Requests unterschiedliche Laenge haben.

Batching braucht ein Latenzbudget. Ein Scheduler gruppiert nach Modell, Phase, Laenge und Prioritaet. Zu grosse Batches verbessern Durchsatz, koennen aber TTFT und Fairness verschlechtern.

NVIDIA Triton beschreibt Dynamic Batching als transparentes Kombinieren einzelner Requests; vLLM kombiniert Continuous Batching mit effizientem KV-Management. Die richtige Konfiguration entsteht durch Lasttests mit echten Laengenverteilungen.

Batch Size	Gemeinsam ausgefuehrte Requests oder Tokens.
Queue Delay	Maximale Wartezeit zum Bilden eines effizienteren Batches.
Throughput	Verarbeitete Tokens oder Requests pro Sekunde.
Fairness	Lange Requests duerfen kurze nicht unbegrenzt verdraengen.

37. Prompt, Prefix und Semantic Caching

Cache nur, was semantisch und sicher wiederverwendbar ist.

Exact Response Caching passt zu deterministischen oder bewusst wiederverwendbaren Antworten. Prompt Prefix Caching spart wiederholte Prefill-Arbeit fuer identische Anfangstokens. Semantic Caching sucht aehnliche Fragen, riskiert aber falsche Wiederverwendung.

Der Cache Key muss Model, Version, Prompt Template, Tools, Policy, Tenant und relevante Datenversionen enthalten. Sonst liefert ein schneller Cache alte oder fremde Wahrheit.

Extended Cache Retention kann Datenschutz- oder Data-Retention-Eigenschaften veraendern. Cache-Architektur ist deshalb Teil von Security und Compliance, nicht nur Performance.

Exact	Nur identische normalisierte Inputs treffen.
Prefix	Wiederverwendung gemeinsamer KV-Praefixe.
Semantic	Aehnliche Inputs teilen potenziell ein Ergebnis.
Version Key	Verhindert Nutzung nach Model- oder Prompt-Aenderung.

38. Quantisierung und Distributed Inference

Reduziere Precision oder verteile das Modell - aber messe Qualitaet.

Quantisierung speichert Gewichte und manchmal Activations mit weniger Bits. Das reduziert Memory und kann Durchsatz steigern, kann aber Qualitaet und Kernel-Kompatibilitaet beeinflussen. Evals entscheiden, nicht der Formatname.

Tensor Parallelism teilt einzelne Layer-Operationen ueber Devices und kommuniziert haeufig. Pipeline Parallelism verteilt Layer-Stufen und kann Bubbles erzeugen. Data Parallelism repliziert Modelle fuer unabhaengige Batches.

Verteilte Inference lohnt sich, wenn das Modell sonst nicht passt oder ein einzelnes Device die Zielrate nicht erreicht. Sie vergroessert jedoch Failure Domain, Netzwerkabhaengigkeit und Scheduling-Komplexitaet.

Tensor Parallel	Eine Operation wird ueber mehrere Devices geteilt.
Pipeline Parallel	Verschiedene Layer-Stufen liegen auf verschiedenen Devices.
Data Parallel	Mehrere Modellkopien verarbeiten unabhaengige Arbeit.
Quantization	Weniger Bits gegen Memory-, Speed- und Qualitaets-Trade-offs.

39. Model Routing und Multi-Model Serving

Nicht jede Aufgabe braucht das groesste Modell.

Ein Router kann nach Aufgabe, Risiko, Sprache, Tenant-Policy, Latenz und Budget waehlen. Kleine Modelle erledigen Klassifikation oder einfache Transformation; groessere Modelle erhalten komplexe oder eskalierte Aufgaben.

Multi-Model Serving konkurriert um GPU-Memory. Dynamic Loading spart Ressourcen, erzeugt aber Cold Starts. LoRA Adapter erlauben Varianten mit geteilten Base Weights. Routing muss geladene Modelle und Adapter-Lokalitaet kennen.

Fallback darf Semantik nicht still veraendern. Definiere, welche Modelle kompatibel sind und wann eine langsamere oder eingeschraenkte Antwort besser als ein anderes Modell ist.

MENTAL MODEL

```
request -> policy + complexity router
              |-- small model: cheap / fast
              |-- large model: hard / high-value
              |-- human or queue: unsafe / unavailable
```

40. Qualitaet, Kosten und Latenz

AI-Systemdesign optimiert ein Dreieck, nicht eine Zahl.

Kosten entstehen aus Input Tokens, Output Tokens, Accelerator-Zeit, Retrieval, Tool-Aufrufen und Wiederholungen. Eine billige einzelne Completion kann teuer werden, wenn sie Aufgaben nicht loest und Agents Schleifen erzeugen.

Miss Cost per Successful Task. Verbinde Model- und Infrastrukturmetriken mit Evals. Ein groesseres Modell kann durch weniger Retries insgesamt billiger sein; ein kleineres mit gutem Retrieval kann dieselbe Qualitaet erreichen.

Budgets gehoeren in den Request: maximale Tokens, Tool Calls, Zeit und Geld. Der Orchestrator entscheidet innerhalb dieser Grenze, statt unbegrenzt auf ein Ergebnis zu hoffen.

Cost / Token	Technische Serving- oder Provider-Kosten.
Cost / Task	Kosten bis zu einem akzeptierten User Outcome.
Quality	Eval-Score auf relevanten Aufgaben und Risiken.
Latency	TTFT, ITL und Gesamtdauer pro Journey.

TEIL 8 / AI APPLICATIONS

Context ist die eigentliche Anwendung

Models sind austauschbare Reasoning Engines. Produktwissen, Berechtigungen, Tools, Memory und Evaluation bilden das dauerhafte System.

41. RAG als Retrieval-System

RAG ist kein Vector-Search-Endpoint, sondern eine Entscheidungs-Pipeline.

Der Ablauf ist Ingestion, Parsing, Chunking, Enrichment, Embedding, Candidate Retrieval, Filtering, Reranking, Context Packing und Generation. Jede Stufe kann Recall oder Precision verlieren.

Permission Filtering muss vor unsicherer Nutzung stattfinden. Ein hochrelevantes fremdes Dokument bleibt verboten. Filter duerfen nicht nur spaet im Prompt angewendet werden, weil Inhalte bereits Logs, Reranker oder Caches erreichen koennten.

RAG braucht Freshness und Delete Semantics. Ein geloeschtes Dokument muss aus Source, Chunks, Vector Index, Caches und abgeleiteten Antworten verschwinden. Versioniere Ingestion und bewahre Provenance.

MENTAL MODEL

```
source -> parse -> chunks -> embeddings -> candidates
                                        -> permission -> rerank -> pack -> model
```

42. Embeddings und Vector Stores

> Vector Search liefert Nachbarn, nicht Beweise.

Embeddings bilden Bedeutung als Vektoren ab. Exact Search liefert perfekten Recall und lineare Kosten. HNSW und IVFFlat tauschen Recall gegen Geschwindigkeit. Filterselektivitaet beeinflusst ANN stark.

pgvector dokumentiert iterative Scans, weil Filter nach einem approximativen Index-Scan sonst zu wenige Treffer uebrig lassen koennen. Partitionierung oder Tenant-lokale Indizes werden bei grossen Multi-Tenant-Systemen wichtig.

Speichere Model, Dimension, Normalisierung, Chunker-Version und Timestamp. Ein Modelwechsel braucht Dual Write oder Backfill, neue Indizes, Eval-Vergleich und kontrollierte Umschaltung.

Recall@K	Anteil relevanter Items in den ersten K Ergebnissen.
Precision@K	Anteil der Top K, der wirklich relevant ist.
Reranker	Teureres Modell ordnet eine kleine Kandidatenmenge neu.
Hybrid	Lexikalische und semantische Signale werden kombiniert.

43. Context Engineering

> Mehr Kontext ist nicht automatisch mehr Wahrheit.

Ein Context Pack braucht Prioritaet, Herkunft, Aktualitaet und Token Budget. Dedupliziere nahe Chunks, diversifiziere Quellen und reserviere Platz fuer Systemregeln, User-Aufgabe und Tool-Ergebnisse.

Untrusted Text muss als Daten markiert bleiben. Prompt Injection versucht, Daten zu Instruktionen zu machen. Trenne System Policy, Developer Intent, User Input, Retrieved Content und Tool Output strukturell.

Context Compression kann lange Quellen zusammenfassen, riskiert aber Informationsverlust. Bewahre Zitate und Source IDs, damit eine Antwort pruefbar bleibt und bei Bedarf auf Originaltext zurueckgreifen kann.

Budget	Maximale Tokens pro Kontextklasse.
Provenance	Woher eine Aussage oder ein Chunk stammt.
Freshness	Wie aktuell die verwendete Darstellung ist.
Diversity	Mehrere unabhaengige Quellen statt redundanter Treffer.

44. Evals als Produktionssystem

Was nicht evaluiert wird, kann nur zufaellig besser werden.

Offline Evals verwenden gelabelte Aufgaben fuer Qualitaet, Sicherheit und Regression. Online Evals beobachten reale Outcomes, Experimente und menschliches Feedback. Beide brauchen stabile Versionen und reprasentative Segmente.

LLM-as-Judge ist skalierbar, aber selbst modellabhaengig. Kalibriere mit menschlichen Labels, mehreren Kriterien und Blindvergleichen. Fuer harte Fakten sind deterministische Checks oder Source Verification besser.

Ein Release Gate verbindet Model, Prompt, Retrieval, Tools und Policy. Canary Traffic vergleicht Qualitaet, Kosten und Latenz. Rollback-Kriterien werden vor dem Release definiert.

MENTAL MODEL

```
dataset + rubric -> candidate system -> scores + traces
        human calibration <- judge models
        production feedback -> new eval cases
```

45. Agents als kontrollierte Schleifen

Ein Agent ist ein Model in einem Zustands- und Tool-System.

Die Kernschleife beobachtet State, waehlt eine Aktion, ruft ein Tool auf, speichert das Ergebnis und entscheidet ueber den naechsten Schritt. Ohne Budgets und Stop Conditions wird daraus eine unendliche, teure Schleife.

Der Orchestrator besitzt Deadline, Token-, Tool- und Geldbudget. Jeder Step bekommt eine eindeutige ID und persistierten Status. Ein Prozesscrash darf nicht denselben externen Side Effect blind wiederholen.

Plane fuer Human Escalation. Unsichere, irreversible oder hochwirksame Aktionen brauchen Approval. Der Agent darf eine Empfehlung erzeugen, aber die Policy entscheidet die Ausfuehrung.

MENTAL MODEL

```
goal -> observe -> plan -> tool -> result -> update state
          ^                                  |
          +--------- budget / stop ------+
```

46. Tools, Capabilities und Security

> Das Model schlaegt vor; ein deterministischer Layer erlaubt.

Tool Schemas begrenzen Form, nicht Berechtigung. Der Server leitet Identitaet und Policy aus vertrauenswuerdigem State ab. Ein Model-Argument wie role=admin darf keine Autoritaet erzeugen.

Verwende Least Privilege, kurzlebige Credentials, Resource Scopes und Approval fuer irreversible Aktionen. Sandbox Code, beschraenke Netzwerkziele und behandle Tool Output als untrusted Input fuer den naechsten Model-Step.

Audit dokumentiert Actor, Policy, Tool, Argumente, Ergebnis und Side Effect ID. Secrets duerfen nicht in Prompts, Traces oder Model Memory gelangen. Redaction ist eine Boundary-Funktion.

Capability	Signierte oder serverseitig abgeleitete konkrete Befugnis.
Approval	Menschliche oder Policy-Freigabe vor einer Aktion.
Sandbox	Begrenzte Ausfuehrungsumgebung fuer untrusted Code.
Egress Policy	Welche externen Ziele erreichbar sind.

47. Memory fuer Agents

Memory ist ein Datenprodukt, kein Chat-Verlauf ohne Ende.

Working Memory lebt in der aktuellen Aufgabe. Episodic Memory bewahrt Ereignisse. Semantic Memory speichert verdichtete Fakten. Procedural Memory beschreibt Vorgehen und Policies. Jede Klasse braucht andere Retention und Trust.

Agents sollten privat schreiben und Wissen explizit promoten. Shared Memory braucht Provenance, Scope, Version, Konfliktregel und Delete Semantics. Ein Vektor allein ist keine Berechtigung.

Memory Consolidation fasst Wiederholungen zusammen und entfernt veraltete Varianten. Diese Hintergrundarbeit darf keine neue Wahrheit erfinden. Bewahre Links auf Quellen und auditierbare Promotionen.

Working	Kurzlebiger State einer Aufgabe.
Episodic	Vergangene Aktionen und Ergebnisse.
Semantic	Verdichtete Aussagen und Wissen.
Procedural	Regeln, Skills und erlaubte Vorgehensweisen.

48. Long-running Workflows

Dauerhafte Agents brauchen Workflow-Semantik, nicht einen ewigen Prozess.

Lang laufende Aufgaben ueberleben Deployments, Timeouts und Restarts. Persistiere Workflow State zwischen Steps. Timer, Retries und externe Callbacks werden zu Events, nicht zu schlafenden Threads.

Exactly-once Side Effects existieren selten. Verwende Step IDs, Idempotency Keys und Compensating Actions. Ein Workflow kann at-least-once ausgefuehrt werden und fachlich effectively-once wirken.

Versioniere Workflow Definitionen. Laufende Instanzen muessen alten Code beenden koennen oder explizit migriert werden. Ein neues Prompt Template darf nicht still die Semantik halb fertiger Tasks veraendern.

Durable State	Persistierter Fortschritt ausserhalb eines Prozesses.
Compensation	Fachliche Gegenaktion statt technischem Rollback.
Timer	Dauerhaft geplantes Zukunftsereignis.
Workflow Version	Kompatibler Codepfad fuer laufende Instanzen.

49. Multi-Agent-Systeme

Mehr Agents erzeugen mehr Koordination, nicht automatisch mehr Intelligenz.

Spezialisierung hilft, wenn Aufgaben klare Rollen, unterschiedliche Tools oder parallele Informationssuche besitzen. Ein Supervisor kann zerlegen und zusammenfuehren. Shared State braucht Ownership und Konfliktpolitik.

Kommunikationskosten wachsen schnell. Agents koennen sich gegenseitig Fehler bestaetigen oder Schleifen erzeugen. Begrenze Fan-out, Tiefe, Budget und gemeinsame Memory Writes. Evaluiere die Gruppe gegen einen starken einzelnen Agenten.

Verwende strukturierte Claims fuer widerspruechliche Aussagen. Confidence allein ist keine Autoritaet. Source Quality und serverseitige Rollen bestimmen, welche Aussage akzeptiert oder zur menschlichen Klaerung eskaliert wird.

MENTAL MODEL

```
supervisor
  |-- researcher -> evidence
  |-- operator   -> tool outcomes
  |-- critic     -> risk review
  +-- synthesizer -> final answer with provenance
```

TEIL 9 / REFERENCE ARCHITECTURES

Fuenf Systeme, dieselben Grundideen

Komponenten aendern sich. Die Muster bleiben: klare Source of Truth, bounded work, sichere Side Effects, gute Queues und messbare User Outcomes.

50. AI Chat SaaS

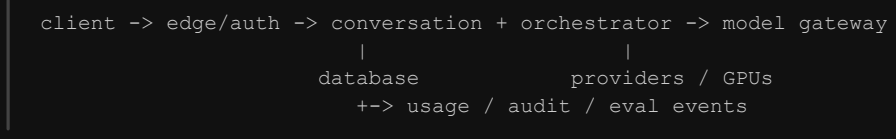
> Der einfache Chat ist ein Multi-Tenant-Streaming-System.

Das Edge authentifiziert User und Tenant, ein Conversation Service speichert Messages, ein Orchestrator baut Context und ruft das Model Gateway. Tokens werden gestreamt; Usage und Audit werden asynchron, aber dauerhaft erfasst.

Ein Model Gateway kapselt Provider, Routing, Budgets, Retries und Versionen. Conversation State bleibt in einer Datenbank; grosse Attachments liegen in Object Storage. Private Daten werden vor Retrieval und Logging gefiltert.

Skalierung beginnt mit stateless APIs und einer relationalen Source of Truth. Spaeter trennen Cells Tenant-Gruppen. Der globale Pfad kennt nur Tenant Placement und Identity-Metadaten.

MENTAL MODEL



51. Knowledge Copilot

> Die schwierige Komponente ist Freshness plus Permission-aware Retrieval.

Connectoren lesen Quellen, schreiben rohe Artefakte in Object Storage und Events in eine Ingestion Queue. Parser und Chunker erzeugen versionierte Chunks. Embedding Worker aktualisieren den Vector Index. ACL-Metadaten bleiben mit jeder Darstellung verbunden.

Der Query Path authentifiziert, filtert Tenant und Permissions, kombiniert lexical und semantic Candidates, rerankt und packt Kontext. Die Antwort zitiert Quellen. Deletes und ACL-Aenderungen haben ein messbares Propagation SLO.

Evals messen Retrieval Recall, Citation Accuracy und Answer Groundedness getrennt. Sonst kann ein gutes Sprachmodell schlechte Suche verdecken.

MENTAL MODEL

```
sources -> connectors -> object store -> parse/chunk -> vector + text index
query   -> auth/ACL -> hybrid retrieve -> rerank -> context -> model -> citations
```

52. Coding Agent

Code ist ein grosser, sich staendig aendernder Graph mit gefaehrlichen Tools.

Ein Workspace Service verwaltet Repository Snapshot, Branch und Sandbox. Indexer bauen Symbol-, Text- und optional Embedding-Sichten. Der Agent liest gezielt, plant kleine Patches, fuehrt Tests in isolierter Umgebung aus und sammelt Diff sowie Logs.

Tool Policies begrenzen Shell, Netzwerk, Secrets und Dateipfade. Writes sind lokal und reviewbar. Push, Merge, Deployment oder externe Nachrichten brauchen explizite Autoritaet.

Lang laufende Tasks persistieren Plan, Tool Calls und Artefakte. Ein Crash kann die Sandbox neu verbinden oder aus einem Snapshot rekonstruieren. Erfolg ist bestandener Test plus akzeptierter Diff, nicht nur eine fluessige Antwort.

Workspace	Isolierter, versionierter Arbeitszustand.
Indexer	Schnelle Projektion fuer Symbole, Text und Abhaengigkeiten.
Sandbox	Begrenzt Code-Ausfuehrung und Side Effects.
Verifier	Tests, Lints und Policy Checks vor Freigabe.

53. Realtime Voice Agent

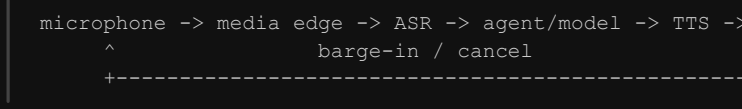
Audio ist ein harter Latenz- und Unterbrechungs-Workflow.

Der Client streamt Audio ueber eine dauerhafte Verbindung. Voice Activity Detection, Speech Recognition, Orchestrator, Model und Text-to-Speech bilden eine Pipeline. Nutzer erwarten Barge-in: neue Sprache muss alte Ausgabe stoppen.

Jede Stufe besitzt kleine Buffers. Zu grosse Buffers erhoehen Latenz, zu kleine verlieren Effizienz. Regionale Media Edges reduzieren RTT. Session State und Tool Calls muessen auch bei Reconnect konsistent bleiben.

Metriken sind End-of-Utterance bis First Audio, Unterbrechungszeit, Transkriptqualitaet, Tool Success und Gesamtkosten pro Minute. Fallback kann auf Text oder eingeschraenkte Funktionen wechseln.

MENTAL MODEL



54. Agent Platform und Control Plane

Die Plattform besitzt Identitaet, Policy, Memory und Audit - nicht jede Agent Runtime.

Externe Runtimes planen und benutzen Tools. Die Control Plane authentifiziert Agents, vergibt Capabilities, speichert erlaubtes Memory, baut Context, misst Usage und auditierte Side Effects.

Der synchrone Pfad bleibt klein: Auth, Policy, bounded Retrieval, Context und Response. Embeddings, Evals, Analytics und Event Delivery laufen ueber dauerhafte Queues. Eine Outbox verbindet Datenbankmutationen mit dem Event Bus.

Bei Wachstum ordnet ein Directory Tenants einer Home Cell zu. Jede Cell besitzt Identity Cache, API, Datenbank, Retrieval und Worker. Globale Systeme duerfen keinen normalen Context Request blockieren.

MENTAL MODEL

```
agent runtimes -> SDK/API -> identity -> policy -> context -> audit
                                  |          |
                              data plane   async events / workers
```

TEIL 10 / EVOLUTION

Baue die naechste Architektur, nicht die letzte

Die beste fruehe Architektur hat klare Grenzen und einen glaubwuerdigen Upgrade-Pfad. Sie simuliert nicht die Komplexitaet eines Unternehmens, das noch nicht existiert.

55. Prototype

Ein Prozess, eine Datenbank, echte Invarianten.

Beginne mit einem modularen Monolithen, einer relationalen Datenbank, Object Storage und einem Background Worker. Verwende Managed Model APIs, bis eigene Inference einen messbaren Produkt- oder Kostenvorteil bietet.

Schon jetzt brauchst du Tenant IDs, serverseitige Auth, Idempotency Keys, Migrationen, strukturierte Logs und Limits. Das sind keine Scale-Features, sondern Korrektheit.

Vermeide fruehe Microservices, Multi-Region Writes und mehrere Datenbanken fuer dieselbe Wahrheit. Jede Netzwerkgrenze vervielfacht Failure Modes und Deployment-Kosten.

Keep	Domain-Grenzen, IDs, Audit, Tests und Datenownership.
Delay	Sharding, eigene GPU-Flotte und globale Active-active Writes.
Measure	User Outcome, Latenz, Kosten und Datenwachstum.

56. Product-Market Fit

Erst jetzt werden Betrieb und Kosten zum Produkt.

Horizontal replizierte APIs, Connection Pooling, CDN, Queue Worker und Read Optimierung reichen oft weit. Fuehre SLOs, On-Call, Capacity Reviews, Backup Restore und Canary Deployments ein.

AI-Kosten werden sichtbar. Route einfache Aufgaben, cache sichere Prefixes, batch asynchrone Arbeit und evaluiere kleinere Modelle. Miss Kosten pro erfolgreicher Aufgabe.

Hot Tenants und unfaire Workloads erscheinen. Quoten, Prioritaeten und Tenant-isolierte Queues kommen vor globalem Sharding. Optimiere den echten Engpass.

MENTAL MODEL

```
stateless APIs -> pooled database -> workers / queues
      |                |                 |
   autoscale       query tuning      bounded retries
```

57. Millionen Nutzer oder Agents

Partitioniere zuerst Ownership, dann Hardware.

Ein Tenant Directory ordnet Kunden einer Region, Cell und Shard-Gruppe zu. Daten und Events erhalten stabile Partition Keys. Grosse Tenants koennen dedizierte Kapazitaet erhalten; kleine teilen standardisierte Cells.

Trenne Online Serving, Batch und Analytics. Der synchrone Pfad darf keine globale Data Warehouse Query oder Training Pipeline brauchen. Cold Data wandert in Object Storage; Serving Views bleiben begrenzt.

GPU Serving wird ein eigener Scheduler- und Capacity-Bereich. Reserviere Basiskapazitaet, scale vor bekannten Peaks und verwende Admission Control fuer unerwartete Bursts.

Directory	Kleine globale Zuordnung von Tenant zu Heimat.
Cell	Wiederholbare, isolierte Capacity Unit.
Dedicated	Eigene Ressourcen fuer sehr grosse oder regulierte Tenants.
Tiering	Hot, warm und cold Data nach Zugriff und Kosten.

58. Globale Plattform

Globales Routing, regionale Wahrheit, lokale Failure Domains.

Edge Routing bringt Requests zur Tenant Home Region. Regionale Cells verarbeiten normale User Journeys autonom. Globale Control Planes verteilen Placement und Konfiguration asynchron und duerfen nicht im Hot Path liegen.

Failover ist eine Autoritaetsentscheidung. Stoppe oder fence den alten Writer, bestimme einen ausreichend aktuellen neuen Primary und aktualisiere Routing. Split Brain ist schlimmer als ein kurzer kontrollierter Ausfall.

Rollouts laufen in Wellen ueber wenige Cells. Automatische Health Gates stoppen frueh. Ein globaler Fehler darf nicht durch simultanes Deployment entstehen.

MENTAL MODEL

```
global edge -> tenant directory cache -> home region -> cell -> shard
                         global control plane ---- asynchronous only
```

59. Der Systemdesign-Entscheidungsrahmen

Beantworte diese Fragen, bevor du eine Komponente hinzufuegst.

Welche User Journey und welches messbare Ziel verbessern wir? Welche Invariante besitzt die Komponente? Was ist ihre Source of Truth? Welche Arbeit ist bounded? Wie sieht der Degradation-Pfad aus?

Welche neue Failure Domain, Queue, Replikation oder On-Call-Arbeit entsteht? Wie wird migriert, getestet, beobachtet und zurueckgerollt? Was kostet sie bei zehnfacher Last?

Kann eine einfachere Aenderung am Datenmodell, Index, Cache Key, Batch oder Produktversprechen dasselbe Problem loesen? Architecture is subtraction before addition.

Why	Welches konkrete Problem und welcher User Impact?
Boundary	Welche Verantwortung wird hier isoliert?
Failure	Was passiert langsam, doppelt, teilweise oder gar nicht?
Evidence	Welche Lasttests, Evals und SLOs beweisen Erfolg?
Exit	Wie wechseln oder entfernen wir die Komponente spaeter?

60. Die zehn Ideen, die bleiben

Produkte und Clouds wechseln; diese Prinzipien bleiben.

Erstens: Beginne mit Invarianten und Lastmathematik. Zweitens: Jede Arbeit braucht eine Grenze. Drittens: Timeouts erzeugen Ungewissheit, Idempotenz macht Wiederholung sicher. Viertens: Datenmodelle bestimmen Moeglichkeiten. Fuenftens: Replikation kopiert auch Fehler.

Sechstens: Queues brauchen Backpressure und Ownership. Siebtens: Isolation reduziert Blast Radius. Achtens: Observability misst User Outcomes, nicht nur Maschinen. Neuntens: AI braucht Evals und Budgets neben klassischer Reliability. Zehntens: Die naechste einfache Architektur ist besser als die letzte komplexe Architektur zu frueh.

Wenn du diese Ideen an einem konkreten System mit Zahlen, Failure Paths und Trade-offs erklaeren kannst, kannst du neue Technologien schnell einordnen. Das ist das eigentliche Ziel von Systemdesign.

MENTAL MODEL

```
promise -> invariant -> model -> bounded path -> failure policy -> evidence

simple first. measured always. isolated when necessary.
```

TEIL 11 / PRIMARY SOURCES

Weiterlesen ohne Umweg

Die Ausgabe ist ein originales Lernbuch. Diese primaeren Dokumentationen und Engineering-Ressourcen vertiefen die wichtigsten modernen Mechanismen.

Kubernetes Autoscaling

https://kubernetes.io/docs/concepts/workloads/autoscaling/

Kubernetes Horizontal Pod Autoscaler

https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

Google SRE Resources

https://sre.google/resources/

AWS Builders' Library - Shuffle Sharding

https://aws.amazon.com/builders-library/workload-isolation-using-shuffle-sharding/

AWS Well-Architected - Cell-based Architecture

https://docs.aws.amazon.com/wellarchitected/latest/reducing-scope-of-impact-with-cell-based-architecture/

vLLM Documentation

https://docs.vllm.ai/

NVIDIA Triton - Dynamic Batching

https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/batcher.html

pgvector

https://github.com/pgvector/pgvector

PostgreSQL - Concurrency Control

https://www.postgresql.org/docs/current/mvcc.html

PostgreSQL - Write-Ahead Log

https://www.postgresql.org/docs/current/wal-intro.html

OAuth 2.0

https://www.rfc-editor.org/rfc/rfc6749

OpenID Connect Core

https://openid.net/specs/openid-connect-core-1_0.html

JWT Best Current Practices

https://www.rfc-editor.org/rfc/rfc8725

Raft Consensus Paper

https://raft.github.io/raft.pdf

OpenTelemetry Concepts

https://opentelemetry.io/docs/concepts/

OpenAI API - Batch

https://platform.openai.com/docs/guides/batch

OpenAI API - Prompt Caching

https://platform.openai.com/docs/guides/prompt-caching

Google Cloud Architecture Center - AI and ML

https://cloud.google.com/architecture/ai-ml

> Dieses Buch ersetzt keine produkt- oder providerspezifische Dokumentation. Sein Zweck ist ein dauerhaftes mentales Modell, mit dem neue Details schneller verstanden und kritisch geprueft werden koennen.